

Building, Characterizing, and Optimizing a Hierarchical Agentic Inference System

Anabyv Esirkiza Gmichail Tanmay

Department of Computer Science and Engineering, University of Washington

June 2026

Summary

Agentic language-model systems achieve strong results on reasoning and tool-use tasks, but they typically lean on a single large model that is expensive and slow. We built and characterized a *hierarchical agentic inference system* that pairs a small, locally-hosted model acting as a self-reflective router with a tier of progressively larger external solver models. A server-side agent loop validates each answer, repairs failing code, and escalates to the strongest model only when the cheaper path fails. On the full 248-problem benchmark the system solved 96.4% of problems, but the striking result is that 99.6% of all inference cost came from the small number of calls that reached the 550B tier, while the cheapest tier solved every one of the 220 problems it received for under half a percent of the cost. A live on-TPU run validates the same pipeline end-to-end, and re-pricing shows that defaulting to the strongest model would cost roughly $2,600\times$ more for no accuracy gain. We also characterized the local router under vLLM and studied solver-side prompt optimizations, finding that token-cutting tricks can be net cost-negative once their prompt overhead and escalation side effects are accounted for. The dominant cost lever is therefore routing accuracy—keeping problems off the expensive tier—rather than router speed or token thrift.

1 Introduction

Traditional LLM interactions are single- or multi-turn chats against one model. Agentic applications, in contrast, decompose a goal into steps, invoke tools, validate intermediate results, and reflect on failures. The dominant implementation choice today is to attach this control flow to a single high-end model: the agent is smart because the underlying model is, and the system simply absorbs the cost. That choice has two consequences. At the *application* level, every problem—trivial or hard—pays the same per-token rate, which is wasteful when most problems are easy. At the *systems* level, the inference engine spends most of its decode budget generating long reasoning traces on a model that is over-provisioned for the task, while small, cheap models sit idle.

This project takes the opposite stance. We treat *which model runs* as the primary control variable of the agent and ask how cheaply we can solve a mixed benchmark if the agent itself is responsible for picking the right solver. The design pattern is *dynamic model orchestration*: a small, locally-hosted model serves as a router that reads the problem, picks the cheapest external solver likely to succeed, and emits a structured decision; a server-owned agent loop then drives that solver through validation, tool-assisted repair, and, if necessary, a single escalation to the strongest available model. The router is itself an agent step—it self-critiques in a second pass and may revise its choice before any expensive solver call is made.

To quantify effectiveness, we introduce an analytical cost function calibrated to inference-engine behavior and apply it to every model call the agent makes. The function attributes cost to the specific decisions the agent took—which model was selected, how many prompt and completion tokens it consumed, how many tool invocations and attempts the problem required, and whether escalation fired—so a run’s aggregate cost can be decomposed and compared against any other run on the same benchmark.

2 System Design and Implementation

This section describes the problem domain we target, the agentic pattern we implement, the tool and state machinery the agent relies on, and the two-tier software architecture that hosts it.

2.1 Problem Domain and Use Case

The target application is a general-purpose autonomous reasoning agent that answers a heterogeneous stream of benchmark problems under a hard constraint on aggregate inference cost. Each problem carries enough metadata for the system to route, solve, and grade it without human intervention (Table 1).

The benchmark assembled for this project contains 248 problems: 164 code problems from HumanEval, 40 math problems from GSM8K, and a curated set of ~ 44 math, reasoning, factual, and image problems. Problems are loaded per domain, so a run can target one domain or their union. The workload is deliberately non-trivial: code problems must execute against unit tests in a sandboxed interpreter, web problems require fetching and conditioning external

Field	Description
<code>problem_id</code>	Unique integer identifier.
<code>problem</code>	The problem text shown to the solver.
<code>difficulty</code>	One of <code>very_easy</code> , <code>easy</code> , <code>medium</code> , <code>hard</code> .
<code>category</code>	One of <code>math</code> , <code>code</code> , <code>factual</code> , <code>reasoning</code> , <code>web</code> , <code>image</code> .
<code>verify</code>	Validation mode: <code>match</code> (exact string), <code>tests</code> (asserts run against the model’s code), <code>judge</code> (LLM-graded), or <code>heuristic</code> (named server-side validator).

Table 1: Per-problem metadata schema.

context before answering, and arithmetic or factual problems range from trivial lookups to multi-step reasoning. A single model choice cannot serve this mix efficiently, so the agent’s job is to land each problem on the cheapest tier that will solve it.

2.2 Agentic Pattern: Dynamic Model Orchestration

We implement dynamic model orchestration as the primary pattern, with reflection layered into both the router and the solver. The agent is realized as two cooperating loops.

Router loop. The locally-hosted Granite-4.1-3B model receives a system prompt listing the four external solver tiers, each annotated with its parameter counts and a plain-English description of the reasoner it represents (high schooler, second-year university student, entry-level worker, PhD student). The router emits a structured decision consisting of a brief reasoning trace and a chosen `model_id`. Because Granite-4.1-3B has no inherent chain-of-thought behavior, requiring it to produce the reasoning field *before* the choice forces a prompt-induced chain-of-thought. A second turn then asks the router to inspect its own reasoning and decide whether the initial rationale was correct, allowing it to revise its decision. The server executes the revised `model_id` and records the updated reasoning. If the decision is unparseable, the router fails safe to the strongest model rather than dropping the problem.

Solver loop. The server’s `solve_problem` routine drives the chosen external model through up to `max_attempts` tries. For each try it (1) prepares tool context (e.g., fetches and truncates the web page for a `web` problem), (2) builds the solver chat with format-specific answer instructions, (3) calls the model, and (4) validates the answer in the mode the problem requires—executing candidate code against its assertions in the sandboxed runner for `verify=tests` problems. On failure, the server appends the runner’s stdout/stderr back to the model so it can repair its code on the next attempt. If all attempts fail, the server escalates exactly once to the strongest registered model and reruns validation. This one-shot escalation cap keeps the worst-case cost bounded.

State and tools. For each problem the agent maintains the running conversation, the tools invoked, the tokens and cost accumulated, and the size of any injected external context. State is held in memory for the duration of the problem and persisted at the end; no agent-side state survives across problems, which keeps the service horizontally scalable. The agent has two deterministic, server-driven tools rather than model-native function calls: `web_search` fetches a problem’s `source_url`, optionally compresses the body, and caps it at 10,000 characters before it enters the prompt; `run_python_code` is a sandboxed executor whose output feeds the code-repair turn.

2.3 Two-Tier Software Architecture

Local inference tier. The local router is IBM Granite-4.1-3B served behind a vLLM OpenAI-compatible endpoint and exercised concurrently by an `asyncio` driver that keeps multiple router requests in flight. This is what lets the engine see a decode-heavy, batchable workload rather than serialized one-at-a-time calls—a prerequisite for the systems-level optimizations we evaluate in Section 4.

External solver tier. Four models are exposed through an OpenAI-compatible provider (OpenRouter in our setup), ordered into progressively stronger tiers primarily by active parameter count and expected reasoning ability (Table 2). This gives the router a discrete set of cost–capability tradeoffs. Active parameter count is the right axis because three of the four solvers are mixture-of-experts models, whose serving cost tracks active rather than total parameters. Post-training also matters when interpreting the tiers: the strongest tier is treated as stronger not only because of its larger scale but because it is expected to have undergone a more extensive reasoning-oriented post-training regime, which is why the gap between the top two tiers is not explained by parameter count alone.

Tier	Model	Total / Active params
Small (cheapest)	OpenAI GPT-OSS-20B	21B / 3.6B
Medium	OpenAI GPT-OSS-120B	117B / 5.1B
Large	Qwen3-Coder	480B / 35B
Strongest	NVIDIA Nemotron-3-Ultra	550B / 55B

Table 2: External solver tiers, ordered by capability. The strongest tier is the one-shot escalation target.

Agent service. A FastAPI service mediates between the two tiers and owns everything that is not inference: tool execution, validation, the cost function, and metrics persistence. Each run is registered ahead of time with a unique `run_id` and a fixed set of optimization flags, which the service treats as the authoritative configuration for everything it logs.

2.4 Optimizations

Each run is registered with a fixed set of boolean optimization flags, grouped by where they take effect:

- **Inference-engine:** `quantized_local_lm` (BF16 $\leftrightarrow$ FP8 weights) and `quantized_kv_cache` (FP8 KV cache), both targeting the local router’s throughput and memory.
- **Prompt and context:** `caveman` (a terse-rewrite instruction meant to cut completion tokens), `web_search_compression` (caps web context size), and two long-context compressors—a deterministic head-and-tail variant and an AI-driven variant that falls back to the deterministic one on any error.
- **Routing policy:** `capabilities_prompt` adds per-model benchmark scores to the router prompt, and `local_model_solves` lets the router short-circuit trivial `very_easy` / `match` problems without any external call.

Flags are independently toggleable against a `baseline` run with all of them disabled, so the cost function can attribute any movement to a single optimization or a stack of them.

3 Performance Characterization Framework

We characterize the system at two levels. The *system level* measures the efficiency of the local routing infrastructure—throughput, latency, and memory of the vLLM-served router. The *application level* measures the effectiveness of the complete agentic workflow—solve rate, routing quality, and the cost–performance tradeoff. The two are linked: a faster but less accurate router can lower local serving cost while *raising* total application cost through extra retries and escalations, so an optimization is only deemed beneficial if it improves the overall cost-vs-quality frontier.

3.1 Cost Model

Because provider API prices are VC-subsidized and would simply reward routing everything to the largest model, we score every model call with an analytical cost function tied to inference-engine behavior. For a call with p prompt tokens and c completion tokens on a model whose active-parameter scale (relative to the smallest solver) is s , the per-call cost combines a quadratic attention prefill, a linear MLP term, and a decode term, modulated by a mild hardware penalty on total size T :

$$C = \underbrace{(\alpha s^2 + (1 - \alpha)s)}_{\text{model factor}} T^{1.2} \cdot [k_{\text{in}} f(p) + k_{\text{out}} c \bar{\ell}], \quad (1)$$

where α is the attention fraction of weights, $f(p)$ is the quadratic prefill term, and $\bar{\ell}$ is the average sequence length during decode. A run’s total cost is the sum of Eq. (1) over every router attempt, solver attempt, repair, and escalation, accumulated per problem. This makes cost an explicit function of the agent’s decisions rather than a flat per-token rate.

3.2 System- and Application-Level Metrics

The router sits on the critical path of every problem, so at the *system level* we report decode throughput (output tokens/s) and serving capacity (requests/s), plus time-to-first-token (TTFT), peak HBM memory (which bounds batch size), and batch efficiency (batched vs. single-request throughput, indicating how well vLLM batches the routing workload). At the *application level* the primary metric is *solve rate*, paired with *aggregate cost* from Eq. (1), plus *escalation rate*, *average attempts* per problem, and router calibration. For calibration we define a target tier per difficulty (`easy` $\rightarrow$ smallest, `medium` $\rightarrow$ mid, `hard` $\rightarrow$ top) and report *router accuracy* (correct initial routes / total) with its two error modes: *over-routing* (routed too high) and *under-routing* (routed too low). Over-routing preserves solve rate but wastes cost on unnecessarily capable models; under-routing is costlier still because it triggers retries and, ultimately, escalation.

4 Performance Evaluation

4.1 Experimental Setup

We draw on four experiments plus a throughput profile, all using the analytical cost function of Eq. (1) in custom units (not provider prices). The **full benchmark** (`baseline_001`) runs all 247 logged problems with real OpenRouter solver calls, difficulty-based routing, and escalation enabled—the only run that exercises the expensive tiers, and thus where we read the cost story. The **live on-TPU** runs (`tpu_bench`) use the real Granite-4.1-3B router served

by vLLM (BF16) on a Google Cloud TPU v5e (16 GB HBM), driving 134 problems end-to-end. The **forced single-model eval** pins each lineup model as a standalone solver (no escalation) over a balanced 32-problem slice to measure real per-model accuracy. The **solver-side study** runs four prompt configurations over the same 66 problems with the router held fixed. Validation uses numeric matching for math, curated validators for reasoning/factual, and **verify=tests** (candidate run against its asserts, with a solve/run/repair loop) for code.

4.2 Full Benchmark: Where the Cost Goes

On the full 247-problem benchmark the agent solved **238 (96.4%)** at an aggregate cost of **0.706** custom units, averaging 1.11 attempts per problem. The interesting result is not the solve rate but *where the cost lives* (Table 3). The router sent 220 problems to the cheapest tier and that tier solved **all 220**, for 0.4% of the run’s cost. Almost the entire 0.706—**99.6%**—came from the **25 calls that reached the 550B tier**, which solved only 16 of them. In other words, the cheap tier carries the workload essentially for free, and the strong tier is simultaneously the cost sink and the place where problems still fail.

Solver tier	Calls	Solved	Cost	% of cost
GPT-OSS-20B (cheap)	220	220	0.0030	0.4%
GPT-OSS-120B (medium)	2	2	0.0002	0.0%
Nemotron-550B (strongest)	25	16	0.7031	99.6%
Total	247	238	0.7064	100%

Table 3: Full-benchmark routing outcome (**baseline_001**, 247 problems). Effectively all cost is incurred by the 25 calls that reached the 550B tier.

Router calibration on this run is good but not perfect: 4.0% of problems were under-routed (escalated because the first choice was too weak) and 2.0% over-routed (an easy problem solved by an unnecessarily expensive tier), leaving $\sim 94\%$ appropriately routed and 10 escalations total. The cost of those errors is visible by difficulty: **easy** problems cost 0.216 in aggregate—*more* than the 0.133 spent on **hard** problems—because a handful of easy code problems were mis-routed or escalated to the 550B tier, each costing 0.03–0.09. This is the concrete form of the thesis: a single mis-routed easy problem on the expensive tier costs more than dozens of correctly-routed hard ones.

4.3 How Much Does the Model Choice Matter?

Two views confirm that the cheap tier is sufficient and that defaulting to a larger model buys nothing but cost. First, re-pricing the live 134-problem workload (same observed tokens) on each solver (Figure 1): the cheapest tier is the cost floor, the medium tier costs $\sim 12\times$, and the 550B model costs $\sim 2,600\times$, with no solve-rate headroom to gain. Second, running each model as a *standalone* solver on a balanced 32-problem slice (Table 4) measures this empirically rather than by projection.

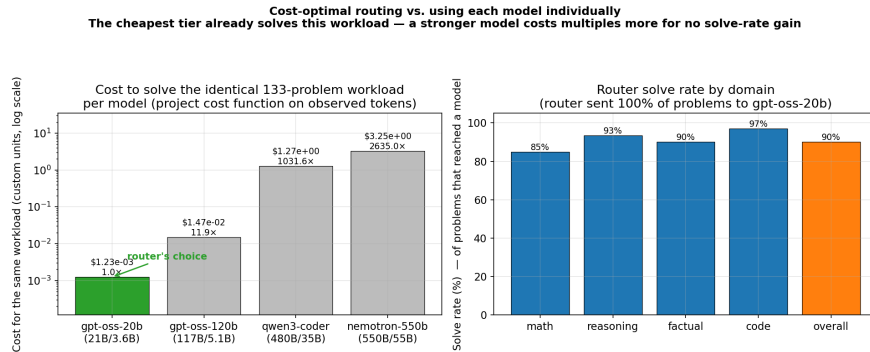


Figure 1: Re-pricing the live 134-problem workload on each model. Left: cost (log scale, custom units) to solve the same workload on each model; the cheapest tier is the floor. Right: the router’s solve rate by domain—uniformly high even though 100% of problems went to the cheapest tier.

The empirical spread is tiny on this mix: GPT-OSS-120B (100%) edges GPT-OSS-20B (96.9%) by ~ 3 points for $\sim 12\times$ the cost per problem, and the strongest model, Nemotron-550B, actually solves *fewer* (95.0%) than the cheap 20B while costing $\sim 1,790\times$ as much per problem and running >120 s/call. The router’s own cost/problem sits essentially at the cheap-tier floor ($\sim 1.4\times$). Its one weak spot is robustness rather than routing: the on-device two-pass granite decision dropped 5/32 problems (non-JSON or timeout) *before* they reached a solver, so end-to-end it trailed simply always using the cheap tier—a router-stage reliability cost, fixable by hardening the JSON parse and gating the self-critique pass on low confidence.

Solver (no escalation)	Solved	Solve rate	Cost/problem
Adaptive router	24/32	88.9% reached	1.3e−5
GPT-OSS-20B only	31/32	96.9%	9.4e−6
GPT-OSS-120B only	32/32	100%	1.1e−4
Nemotron-550B only	19/20	95.0%	1.7e−2
Qwen3-Coder only	—	n/a (404/429)	—

Table 4: Each lineup model run as a standalone solver (escalation disabled) on the same balanced 32-problem slice (Nemotron, slow at >120s/problem, on a 20-problem subset). `qwen3-coder:free` was delisted (404/429) at measurement time—invoked but unreachable.

4.4 Live On-TPU Validation

The `tpu_bench` runs confirm the full pipeline works on real hardware: the Granite router on the TPU v5e drove 134 problems across four domains end-to-end through OpenRouter solvers, solving **121**. Because this sample skews easy/medium, the router sent every reached problem to the cheapest tier with **zero escalations**, and that tier solved every problem it received; the 13 non-solves were all free-tier rate-limits (HTTP 429) or one non-JSON router reply—infrastructure, not capability. This is the existence proof that the on-device router, resolution server, validation, cost accounting, and metrics work together live; the cost spread it cannot exercise is exactly what the full-benchmark run (Section 4.2) provides.

4.5 System Level: Router Serving Throughput

Cost characterizes what we pay per solved problem; throughput characterizes how fast the on-device router serves. We profiled the live BF16 router with a synthetic fixed-length workload (48 requests, concurrency 6, 512-token prompts, 256 output tokens with `ignore_eos` so every request emits exactly `max_tokens`). It sustains ~ 590 **output tok/s** at concurrency 6 (~ 101 tok/s per request), **2.30 requests/s**, a 2.57s p50 latency, and a ~ 46 **ms TTFT**. Because the router only emits a short JSON decision, its throughput matters mostly as latency overhead before a solver is dispatched (~ 46 ms TTFT plus ~ 2.5 s for the two-pass decision). On TPU v5e, `DTYPE=fp8` maps to INT8 weight quantization (v5e has no FP8 weights) while FP8 *KV cache* is supported; the quantization sweep restarts the router per config and is pending a maintenance window on the shared host.

4.6 Solver-Side Optimization Study

The live runs measure *routing*. To isolate *solver-side prompt* optimizations on a clean, comparable matrix, we ran four configurations over the same 66 problems, holding the router fixed as a deterministic difficulty→model stand-in so the only variable is the prompt treatment. To keep the optimization signal from being drowned out, the top tier’s escalation cost was re-tuned so a single escalation costs $\sim 2.7\times$ the large tier rather than $\sim 99\%$ of a run. Table 5 and Figure 2 summarize the outcome.

Config	Solve rate	Escalations	Cost	Δ vs. baseline
baseline	97.0%	3	0.196	—
longctx_ai	98.5%	3	0.189	−3.6%
caveman	97.0%	6	0.536	+174%
combo	97.0%	5	0.486	+148%

Table 5: Solver-side optimization study (4 configs $\times$ 66 problems). Solve rate is statistically flat; the entire signal is in cost.

The headline is that the seemingly obvious token-cutting optimization is the worst one. **caveman** rewrites the solver prompt to demand terse answers and does shrink completion tokens by $\sim 26\%$ as designed—but it is **net cost-negative** (+174%) on this short-problem benchmark (Table 5). The **combo** that includes it inherits the loss, while **longctx_ai** is the only configuration that comes in below baseline, and only marginally.

Why does a token-cutting optimization *raise* cost? Figure 2 gives the answer. The left panel decomposes token usage: **caveman**’s instruction overhead more than doubles *prompt* tokens, swamping the completion-token savings, and its terseness breaks answer-format compliance, which roughly doubles escalations to the expensive top tier. The right panel separates the two effects: non-escalated cost rises from 0.127 to 0.227 (+78%, pure prompt overhead), while escalated cost rises from 0.069 to 0.309 (+349%, more problems bouncing to the top tier). Escalated problems are 58% of **caveman**’s total cost versus 35% for the baseline.

The damage concentrates on the cheapest problems, where the baseline was nearly free. The canonical failure is a **very_easy** “2+2”-class problem the baseline solves instantly for $\sim \$2\times 10^{-6}$: under **caveman** the terse reply fails the exact-match check, escalates all the way to the top tier, still fails the format check, and ends up costing $\sim \$0.038$ and unsolved—a roughly 3,700 $\times$ blow-up on a problem that should be free.

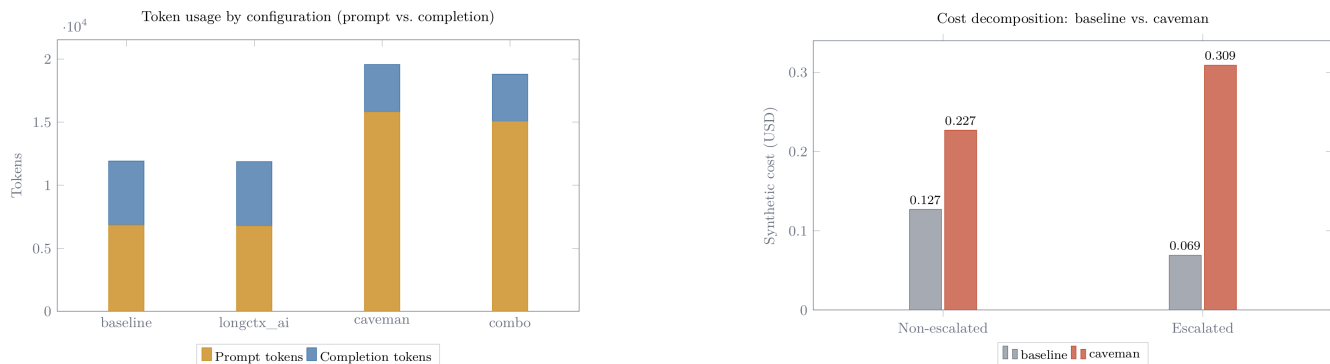


Figure 2: Left: prompt vs. completion tokens by configuration—**caveman** trades a modest completion-token cut for a large prompt-token increase. Right: cost decomposition, baseline vs. **caveman**—the damage is on both axes, prompt overhead on non-escalated problems and far more escalations.

5 Optimization Opportunities

The evaluation points to a clear ordering of where effort pays off.

Routing accuracy dominates everything. The solver tier drives both cost and latency: on the full benchmark 99.6% of all cost came from the 25 calls that reached the 550B tier (Table 3), and re-pricing shows a $\sim 2,600\times$ spread between the cheapest and strongest solver (Figure 1), whereas router serving overhead is milliseconds. The single largest deployment win is therefore avoiding unnecessary escalation and over-routing, not making the router faster. The **caveman** study reinforces this from the opposite direction: an optimization that is locally sensible (fewer completion tokens) is globally harmful because it perturbs routing and escalation behavior.

Optimize for the workload, not the token. **caveman**’s fixed per-call prompt overhead looks worst exactly where problems are smallest, and **longctx_ai** never crosses its compression threshold on short prompts, making it an honest no-op here. Both signs could flip on a long-context workload—the lesson is that prompt optimizations must be matched to the problem distribution, and gated (e.g., by length or difficulty) rather than applied unconditionally.

Router-tier systems wins are second-order. On the local tier, weight + KV-cache quantization (the pending sweep) frees HBM for a larger `max_num_seqs`, prefix caching already amortizes the identical routing prompt across problems, and the two-pass self-revision could be gated on low confidence to halve router latency. These help latency and serving density but, per the first point, are second-order to routing accuracy for total cost. Once routing is accurate and the router is quantized and batched, the remaining cost is intrinsic solver compute on genuinely hard problems—the next frontier being techniques like VLM-describe routing that convert expensive image tokens to a cheap text description before the large model sees them.

6 Conclusion

We built and characterized a hierarchical agentic inference system in which a small, locally-hosted model routes each problem to the cheapest external solver likely to succeed, backed by a server-owned loop that validates, repairs, and escalates within a bounded budget. On the full 248-problem benchmark it solved 96.4% of problems, and the cost decomposition is unambiguous: the cheapest tier solved all 220 problems it received for 0.4% of the cost, while 99.6% of the cost came from the few calls that reached the 550B tier—which also accounted for most of the remaining failures. A live on-TPU run validated the same pipeline end-to-end, a forced single-model study showed the capability spread between tiers is only a few points on this mix, and a controlled study showed that token-cutting prompt optimizations can be net cost-negative once prompt overhead and escalation side effects are counted. Together these reinforce that routing accuracy—keeping problems off the expensive tier—rather than token thrift or router speed is the dominant cost lever. The main limitation is that our benchmark skews easy/medium, so the expensive tiers are exercised mainly by mis-routes; the clear next step is a harder problem set that forces capability-driven escalation, together with the pending router quantization sweep and VLM-describe routing for the image tier.